UNIT-5	ANDROID USER INTERFACE DESIGN	3

User Interface Screen elements, Designing User Interfaces with Layouts, Drawing and Working with Animation. Testing Android applications, Publishing Android application.

USER INTERFACE SCREEN ELEMENTS

Layouts in Android is used to define the user interface that holds the UI controls or widgets that will appear on the screen of an android application or activity screen. Generally, every application is a combination of View and ViewGroup.

An android application contains a large number of activities and we can say each activity is one page of the application. So, each activity contains multiple user interface components and those components are the instances of the View and ViewGroup. All the elements in a layout are built using a hierarchy of View and ViewGroup objects.

Understanding Screen Components

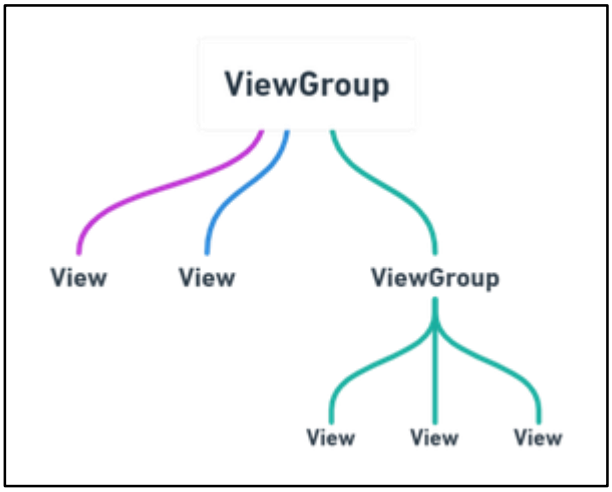
The basic unit of android application is the activity. A UI is defined in an xml file. During compilation, each element in the XML is compiled into equivalent Android GUI class with attributes represented by methods.

View

A View is defined as the user interface which is used to create interactive UI components such as TextView, ImageView, EditText, RadioButton, etc., and is responsible for event handling and drawing. They are Generally Called Widgets.

ViewGroup

A ViewGroup act as a base class for layouts and layouts parameters that hold other Views or ViewGroups and to define the layout properties. They are Generally Called layouts.



The Android framework will allow us to use UI elements or widgets in two ways:

- Use UI elements in the XML file
- Create elements in the Kotlin file dynamically

Types of Android Layout

1) **Android Linear Layout:** LinearLayout is a ViewGroup subclass, used to provide child View elements one by one either in a particular direction either horizontally or vertically based on the orientation property.
2) **Android Relative Layout:** RelativeLayout is a ViewGroup subclass, used to specify the position of child View elements relative to each other like (A to the right of B) or relative to the parent (fix to the top of the parent).
3) **Android Constraint Layout:** ConstraintLayout is a ViewGroup subclass, used to specify the position of layout constraints for every child View relative to other views present. A ConstraintLayout is similar to a RelativeLayout, but having more power.

4) **Android Frame Layout:** FrameLayout is a ViewGroup subclass, used to specify the position of View elements it contains on the top of each other to display only a single View inside the FrameLayout.
5) **Android Table Layout:** TableLayout is a ViewGroup subclass, used to display the child View elements in rows and columns.
6) **Android Web View:** WebView is a browser that is used to display the web pages in our activity layout.
7) **Android ListView:** ListView is a ViewGroup, used to display scrollable lists of items in a single column.
8) **Android Grid View:** GridView is a ViewGroup that is used to display a scrollable list of items in a grid view of rows and columns.

DESIGNING USER INTERFACES WITH LAYOUTS

A layout defines the structure for a user interface in your app, such as in an activity. All elements in the layout are built using a hierarchy of View and ViewGroup objects. A View usually draws something the user can see and interact with. A ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects, as shown in figure 1.

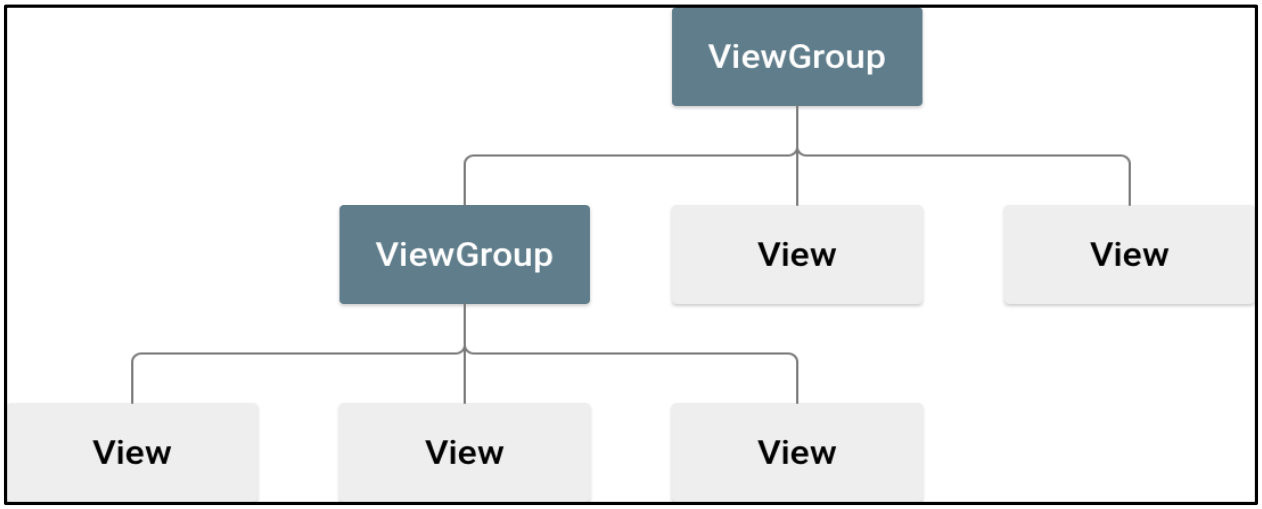


Figure 1. **Illustration of a view hierarchy, which defines a UI layout.**

View objects are often called widgets and can be one of many subclasses, such as Button or TextView. The ViewGroup objects are usually called layouts and can be one of many types that provide a different layout structure, such as LinearLayout or ConstraintLayout.

You can declare a layout in two ways:

1. **Declare UI elements in XML -** Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.
2. **Instantiate layout elements at runtime -** Your app can create View and ViewGroup objects and manipulate their properties programmatically.

Layout parameters

XML layout attributes named layout_something define layout parameters for the View that are appropriate for the ViewGroup it resides in.

Every ViewGroup class implements a nested class that extends ViewGroup.LayoutParams. This subclass contains property types that define the size and position of each child view, as appropriate for the view group. As shown in figure 2, the parent view group defines layout parameters for each child view, including the child view group.

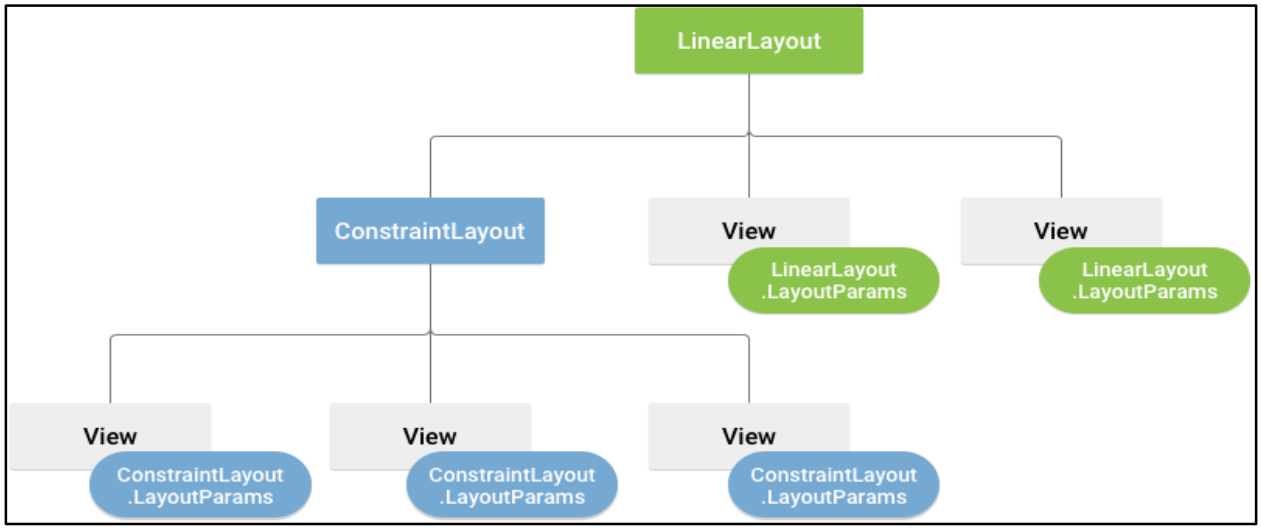


Figure 2. **Visualization of a view hierarchy with layout parameters associated with each view.**

Every LayoutParams subclass has its own syntax for setting values. Each child element must define a LayoutParams that is appropriate for its parent, though it might also define a different LayoutParams for its own children.

All view groups include a width and height, using layout_width and layout_height, and each view is required to define them. Many LayoutParams include optional margins and borders.

You can specify width and height with exact measurements, but you might not want to do this often. More often, you use one of these constants to set the width or height:

- **wrap_content:** tells your view to size itself to the dimensions required by its content.
- **match_parent:** tells your view to become as big as its parent view group allows.

Write the XML

Each layout file must contain exactly one root element, which must be a View or ViewGroup object. After you define the root element, you can add additional layout objects or widgets as child elements to gradually build a View hierarchy that defines your layout.

For example, here's an XML layout that uses a vertical LinearLayout to hold a TextView and a Button:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical" >
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
```

```
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```

Load the XML resource

When you compile your app, each XML layout file is compiled into a View resource. Load the layout resource in your app's Activity.onCreate() callback implementation.

Do so by calling setContentView(), passing it the reference to your layout resource in the form: R.layout.layout_file_name. For example, if your XML layout is saved as main_layout.xml, load it for your Activity as follows:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_layout);
}
```

The Android framework calls the onCreate() callback method in your Activity when the Activity launches.

DRAWING AND WORKING WITH ANIMATION

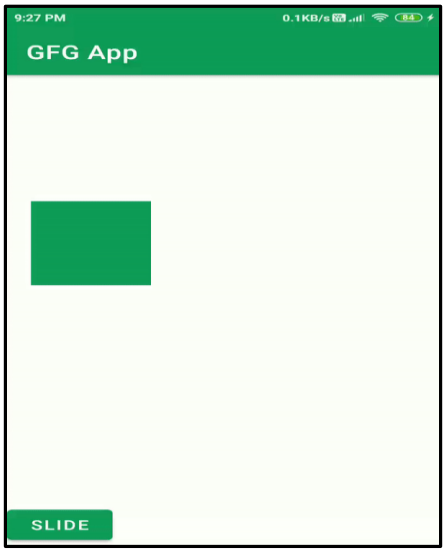
Animation is the process of adding a motion effect to any view, image, or text. With the help of an animation, you can add motion or can change the shape of a specific view. Animation in Android is generally used to give your UI a rich look and feel. The animations are basically of three types as follows:

1. **Property Animation**
2. **View Animation**
3. **Drawable Animation**

1) Property Animation

Property Animation is one of the robust frameworks which allows animation almost everything. This is one of the powerful and flexible animations which was

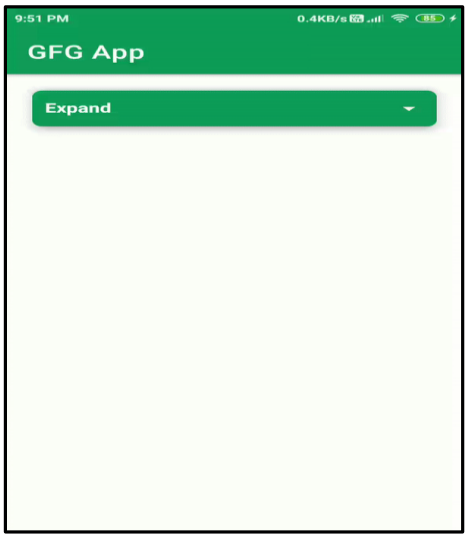
introduced in Android 3.0. Property animation can be used to add any animation in the CheckBox, RadioButtons, and widgets other than any view.



2) View Animation

View Animation can be used to add animation to a specific view to perform tweened animation on views. Tweened animation calculates animation information such as size, rotation, start point, and endpoint. These animations are slower and less flexible.

An example of View animation can be used if we want to expand a specific layout in that place we can use View Animation. The example of View Animation can be seen in Expandable RecyclerView.



3) Drawable Animation

Drawable Animation is used if you want to animate one image over another. The simple way to understand is to animate drawable is to load the series of drawable one after another to create an animation.

A simple example of drawable animation can be seen in many apps Splash screen on apps logo animation.

Important Methods of Animation

Methods	Description
startAnimation()	This method will start the animation.
clearAnimation()	This method will clear the animation running on a specific view.

Steps Involved in Creating Android Animation

Step 1: Create a New Project

Step 2: Working with the strings.xml file

Step 3: Working with the activity_main.xml file

Step 4: Create 6 different types of animation for ImageView

Step 5: Working with the MainActivity.java file

Android Animation Example

Android provides a large number of classes and interface for the animation development. Most of the classes and interfaces are given in android.animation package.

Android Animation enables you to change the object property and behavior at run time. There are various ways to do animation in android.

The AnimationDrawable class provides methods to start and end the animation. Even, you can use time based animation.

MainActivity class

File: MainActivity.java

```
package com.javatpoint.animation;

import android.os.Bundle;
import android.app.Activity;
import android.graphics.drawable.AnimationDrawable;
import android.widget.ImageView;

public class MainActivity extends Activity {
   ImageView anm;
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.logo);
        anm = (ImageView)findViewById(R.id.anm);
        anm.setBackgroundResource(R.drawable.animation);
      // the frame-by-frame animation defined as a xml file within the
drawable folder

        /*
         * NOTE: It's not possible to start the animation during the onCreate.
         */
      }
    public void onWindowFocusChanged (boolean hasFocus) {
        super.onWindowFocusChanged(hasFocus);
        AnimationDrawable frameAnimation =
           (AnimationDrawable) anm.getBackground();
        if(hasFocus) {
           frameAnimation.start();
```

```
            } else {
                frameAnimation.stop();
            }
        }
    }
```

activity_main.xml

Have a view only.

File: activity_main.xml

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingBottom="@dimen/activity_vertical_margin"
        android:paddingLeft="@dimen/activity_horizontal_margin"
        android:paddingRight="@dimen/activity_horizontal_margin"
        android:paddingTop="@dimen/activity_vertical_margin"
        tools:context=".MainActivity" >
        <View />
    </RelativeLayout>
```

File: animation.xml

```
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame0" android:duration="120" />
    <item android:drawable="@drawable/frame1" android:duration="120" />
    <item android:drawable="@drawable/frame2" android:duration="120" />
```

```
    <item android:drawable="@drawable/frame3" android:duration="120" />
</animation-list>
```

TESTING ANDROID APPLICATIONS

Testing is an essential part of the Android app development process. It helps to ensure that the app works as expected, is bug-free, and provides a seamless user experience.

Android testing is an essential part of the app development process. It helps to ensure that the app works as expected, is bug-free, and provides a seamless user experience. There are various testing tools and frameworks available for Android, including JUnit, Espresso, UI Automator, Robolectric, and Mockito.

Android offers various testing tools and frameworks that can be used to write and execute different types of tests, including unit tests, integration tests, and UI tests.

Types of Android Tests

1. **Unit Tests:**
   - Unit tests are automated and are run each time the code is changed to ensure that new code does not break existing functionality.
   - Run multiple tests at once to find and fix problems faster.
   - Test on real Android devices to ensure your app works perfectly on real hardware.
2. **Integration Tests:**

- Integration testing is testing the interface between two software units or modules. It focuses on determining the correctness of the interface.
- Integration testing is a software testing technique that focuses on verifying the interactions and data exchange between different components or modules of a software application.
- Work with your team in real time while testing your app on real devices, discussing issues as you see them.

3. **UI Tests:**
    - Make sure your app looks and works great on all screen sizes and devices
    - Automatically check if your app's design stays consistent even after changes
    - Test your app for accessibility, making sure everyone can use it easily.

Testing Tools and Frameworks

1) **LambdaTest:** It is an AI-powered test orchestration and execution platform that allows you to run Android app tests on an online device farm of real and virtual Android smartphones. You can automate Android tests with mobile app testing frameworks like Appium and Espresso across different Android devices and their OS versions.
2) **JUnit:** This is a popular testing framework for Java that can be used to write unit tests for Android apps.
3) **Espresso:** This is a testing framework for Android that can be used to write UI tests for apps. It provides a fluent API for interacting with UI components and verifying their behavior.
4) **UI Automator:** This is another testing framework for Android that can be used to write UI tests. It provides a set of APIs for interacting with UI components and verifying their behavior.
5) **Robolectric:** This is a testing framework for Android that can be used to write unit tests for Android apps. It allows developers to run tests on the local machine without the need for an emulator or a physical device.
6) **Mockito:** This is a mocking framework for Java that can be used to create mock objects for unit testing.

Steps Involved in Testing Android Applications

Step-by-Step Implementation

Step 1: Create a New Project in Android Studio

Step 2: Working with the activity_main.xml file

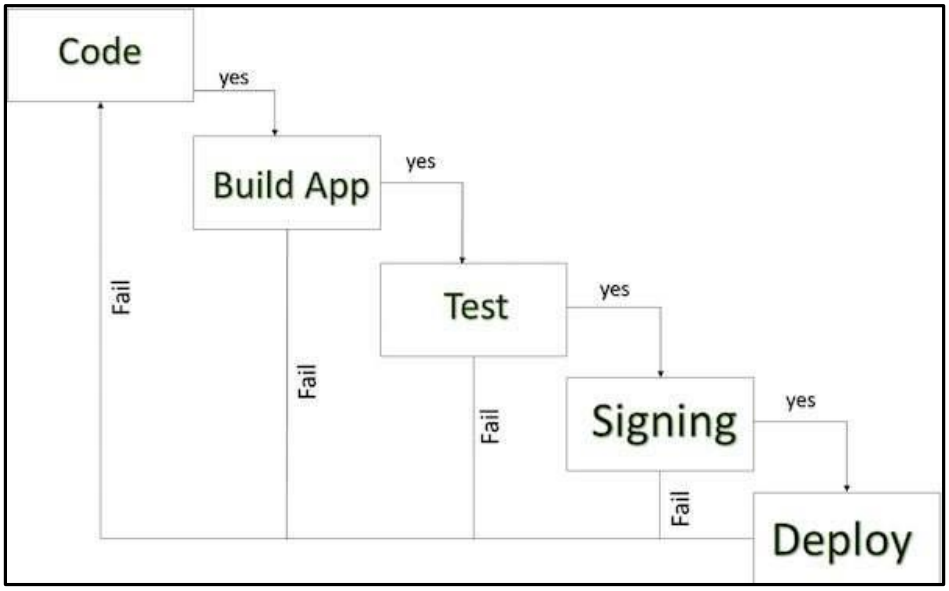
Navigate to the app > res > layout > activity_main.xml

Step 3: Working with the MainActivity File

Step 4: Working with the MainActivity Unit Testing File

PUBLISHING ANDROID APPLICATION

Android application publishing is a process that makes your Android applications available to users. Infect, publishing is the last phase of the Android application development process.



Once you developed and fully tested your Android Application, you can start selling or distributing free using Google Play (A famous Android marketplace).

You can also release your applications by sending them directly to users or by letting users download them from your own website. You can check a detailed publishing process at Android official website.

Publishing is the general process that makes your Android app available to users. When you publish an Android app, you do the following:

- **Prepare the app for release.**

  During the preparation step, you build a release version of your app.

- **Release the app to users.**

  During the release step, you publicize, sell, and distribute the release version of your app, which users can download and install on their Android-powered devices.

1) Prepare your app for release

Preparing your app for release is a multistep process involving the following tasks:

- Configure your app for release.
- Build and sign a release version of your app.
- Test the release version of your app.
- Update app resources for release.
- Prepare remote servers and services that your app depends on.
  Release your app to users

2) Release your app to users

You can release your Android apps several ways. Typically, you release apps through an app marketplace such as Google Play. You can also release apps on your own website or by sending an app directly to a user.

- Release through an app marketplace
- Release your apps on Google Play
    - Prepare promotional materials.
    - Configure options and uploading assets.
    - Publish the release version of your app.